Ch01 導論

1.1 程式語言：驅動現代世界的基礎能力

在數位化浪潮席捲全球的今天，程式語言已不再是軟體工程師的專利，而是如同閱讀與寫作一樣，成為一種橫跨各領域的**基礎素養**與**核心能力**。它是一種教我們如何思考的工具，一種將創意具現化的力量。

> **「這個國家的每一個人都應該學習如何寫程式，因為它教你如何思考。」**
>
> — 史蒂夫·賈伯斯 (Steve Jobs), 蘋果公司聯合創辦人

學會寫程式，意味著你掌握了與機器溝通、自動化解決問題，以及創造新價值的關鍵能力。

1.1.1 程式碼：日常生活中的隱形架構

我們常常忽略，程式碼早已像空氣和水一樣，滲透到生活的每個角落，支撐著現代社會的運作。這些由程式驅動的系統，在我們看不見的地方，默默地發揮作用：

1.1.2 程式語言在各大領域的應用

程式設計的價值體現在它能為各行各業賦能，解決該領域的獨特挑戰：

- **科學研究**：從物理學的粒子碰撞模擬、生物資訊學的基因序列分析，到氣象學的氣候變遷模型，研究人員利用 Python、R 等語言處理龐大實驗數據，驗證科學理論，加速研究進程。

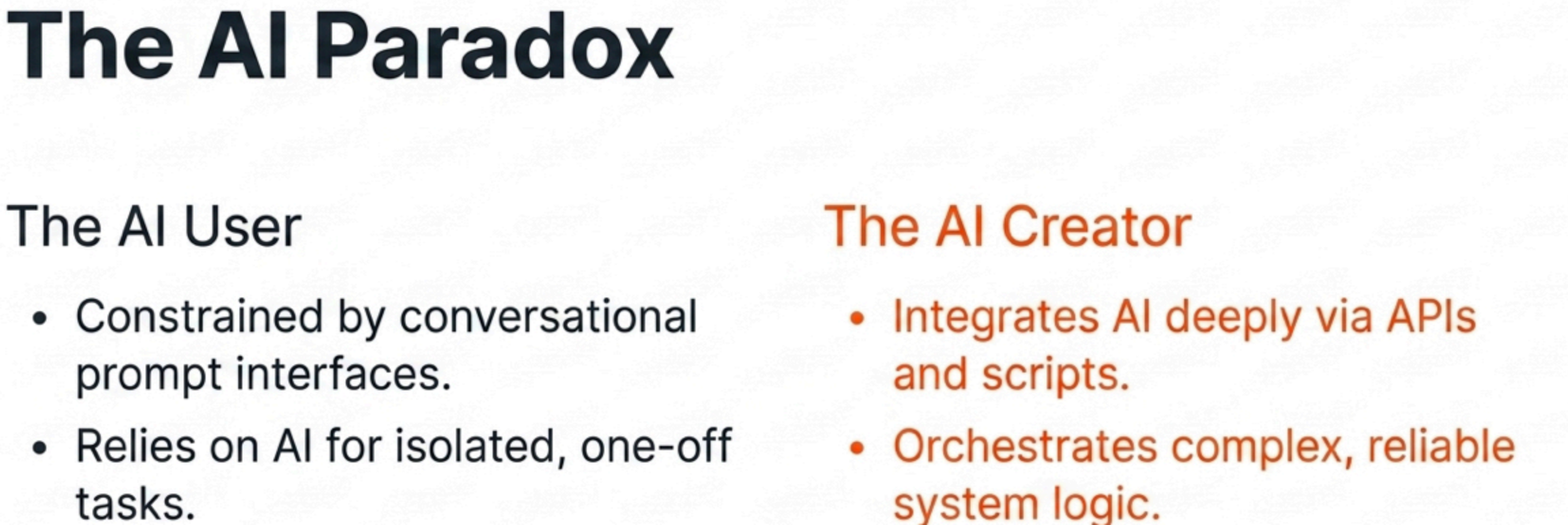
- **商業與金融**：華爾街的量化分析師透過程式執行高頻交易、進行風險評估；行銷專家利用程式自動抓取社群數據，分析消費者行為，實現精準行銷；企業也透過程式自動化處理繁瑣的報表與流程，大幅提升營運效率。

- **藝術與設計**：程式碼成為新時代的畫筆。設計師與藝術家利用 Processing、p5.js 等工具創作互動裝置藝術與生成藝術 (Generative Art)；在動畫或遊戲產業，程式設計師與美術設計師緊密合作，打造出絢麗的視覺特效與栩栩如生的虛擬世界。

- **人文社會科學**：學者透過「數位人文 (Digital Humanities)」方法，撰寫程式來分析大量的歷史文獻、文本資料，從中挖掘新的歷史觀點或社會趨勢。例如，分析數百年的報紙典籍，找出特定詞彙的演變脈絡。

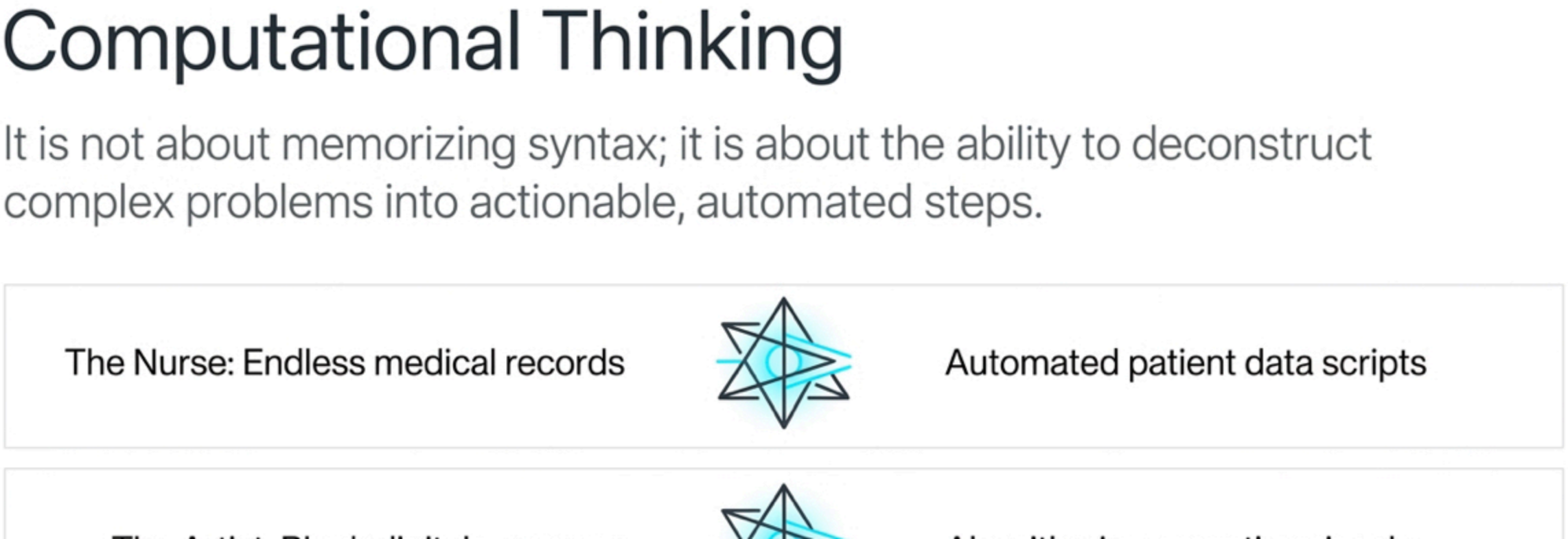
1.1.3 AI 時代，為何程式語言更加重要？

許多人認為，隨著 ChatGPT 這類強大 AI 的出現，似乎只要會「下指令 (Prompting)」，就不再需要寫程式了。這其實是一個誤解。在 AI 時代，程式語言的重要性不減反增，它將你從單純的 **AI 使用者**，提升為能夠駕馭 AI 的**創造者**。



1.1.4 運算思維 (Computational Thinking) 的四大基石

學習寫程式，最核心的收穫並不是背誦特定語法，而是培養**運算思維 (Computational Thinking)**。運算思維是一種結構化的思考方法，幫助我們在面對任何領域的複雜問題時，都能有條不紊地找出精準的解決方案。



1.2 Python 程式設計入門

1.2.1 為何選擇 Python？

Python 由吉多·范羅蘇姆 (Guido van Rossum) 於 1989 年聖誕節期間創造。他期望打造一個兼具 ABC 語言的優雅強大，又避免其封閉性的新語言。Python 的命名靈感則來自他喜愛的 BBC 喜劇《蒙提·派森的飛行馬戲團》(Monty Python's Flying Circus)。



1.2.2 The Zen of Python (Python 之禪)

Python 不僅是一門程式語言，更包含了一套優雅的工程哲學。在任何 Python 終端機中輸入 `import this`，就能印出由提姆·彼特斯 (Tim Peters) 撰寫的《Python 之禪》(The Zen of Python)：

```
import this
```

> **優美優於醜陋 (Beautiful is better than ugly.)**
> **明白優於隱晦 (Explicit is better than implicit.)**
> **簡單優於複雜 (Simple is better than complex.)**
> **複雜優於凌亂 (Complex is better than complicated.)**
> **扁平優於巢狀 (Flat is better than nested.)**
> **稀疏優於緊密 (Sparse is better than dense.)**
> **可讀性很重要 (Readability counts.)**
> **面對錯誤絕不姑息 (Errors should never pass silently.)**

這套哲學時刻提醒開發者：**程式碼被閱讀的次數，遠多於被撰寫的次數。** 寫出結構清晰、易讀易懂的程式碼，是每一位 Python 工程師的終極追求。

1.2.3 直譯語言 vs 編譯語言 (Interpreted vs Compiled)

為了深刻理解 Python 的運作本質，我們需要了解電腦是如何讀懂人類寫下的程式碼。電腦硬體 (CPU) 只能識別二進制的機器碼 (`0` 與 `1`)，將人類可讀的原始碼轉換為機器碼的方式主要分為兩大陣營：

```
【編譯語言 (如 C/C++)】:
原始碼 (hello.c) ──[ 編譯器 Compiler ]──> 執行檔 (hello.exe / 機器碼) ──> CPU 高
```

```
【直譯語言 (如 Python)】:
原始碼 (hello.py) ──> 位元組碼 (.pyc) ──[ 直譯器 Python VM ]──> 逐行轉譯並即時執行
```

比較維度	編譯語言 (Compiled Languages)	直譯語言 (Interpreted Languages)
代表語言	C, C++, Rust, Go, Swift	Python, JavaScript, Ruby, PHP
轉換時機	程式執行前，編譯器一次性將全部程式碼翻譯成機器碼執行檔	程式執行時，直譯器 (PVM) 邊讀取邊逐行轉譯執行
執行效能	**極高**（直接與 CPU 硬體溝通）	**中等**（多了一層虛擬機轉譯開銷）
開發調錯速度	較慢（修改後需重新編譯整個專案）	**極快**（隨改隨跑，支援互動式 REPL 即時測試）

1.2.4 隨堂測驗 (CCQ 1)

問題

下列關於「編譯語言 (如 C++)」與「直譯語言 (如 Python)」特性的比較敘述，何者正確？

A) 直譯語言在執行前必須先花費數分鐘編譯產生 `.exe` 二進位執行檔才能運行。
B) 編譯語言通常執行效能極高，但修改程式碼後必須重新編譯；Python 則支援逐行直譯，具備隨改隨測與極佳的跨平台開發彈性。
C) Python 直譯器可以直接讓硬體 CPU 執行純英文字串，完全不需經過任何轉譯過程。
D) 編譯語言天生不具備型別檢查機制，直譯語言則在編譯期即鎖死型別。

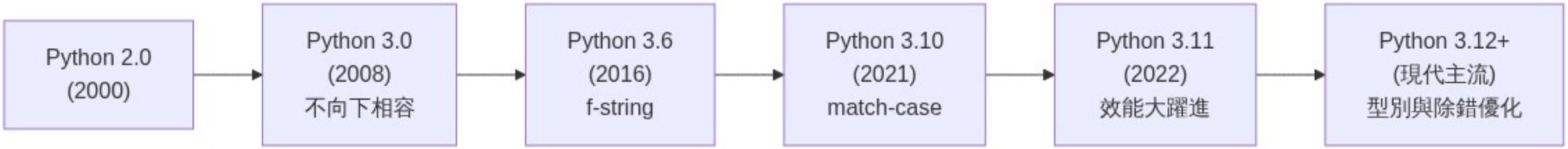
▶ 點擊查看【隨堂測驗】答案與解析

1.3 開發環境的選擇與安裝

要開始撰寫程式，我們需要一個可以編輯並執行 Python 程式碼的環境。這就像寫作需要 Word 或 Pages 一樣。在進入實作安裝前，我們先來了解 Python 的版本歷程與現今豐富的開發工具生態系。

1.3.1 Python 版本演進與選擇

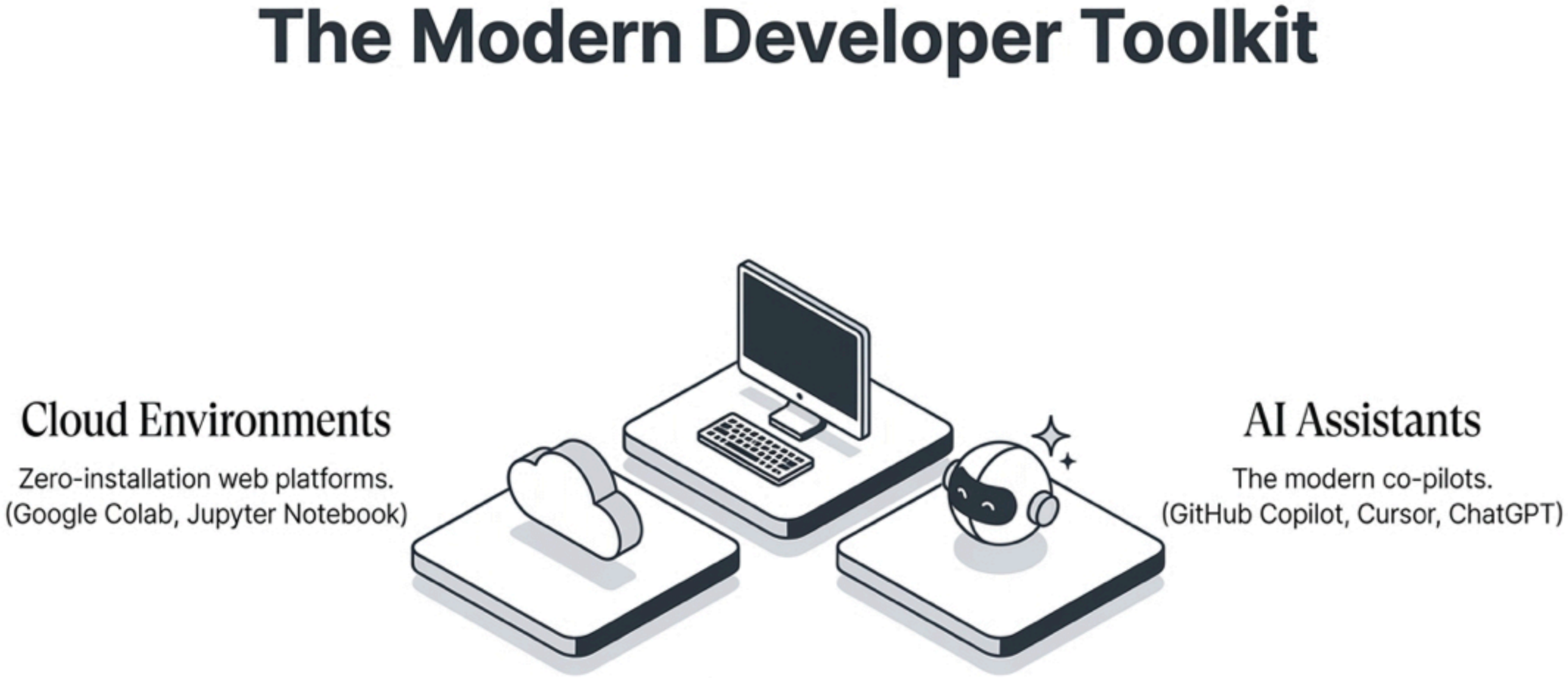
Python 自 1989 年問世以來，經歷了數個重大里程碑。了解這些演進有助於你在學習與撰寫程式碼時，選擇最合適且現代的語法標準：



- **Python 2.x 與 Python 3.0 的分水嶺 (2008 年)**：
  - **不向下相容**：Python 3.0 是語言架構的重大重構，解決了歷史包袱（例如字串預設為 Unicode 解決編碼問題、`print` 成為函式需加括號等）。Python 2 已於 2020 年正式除役 (EOL)，現在所有專案皆採用 Python 3。

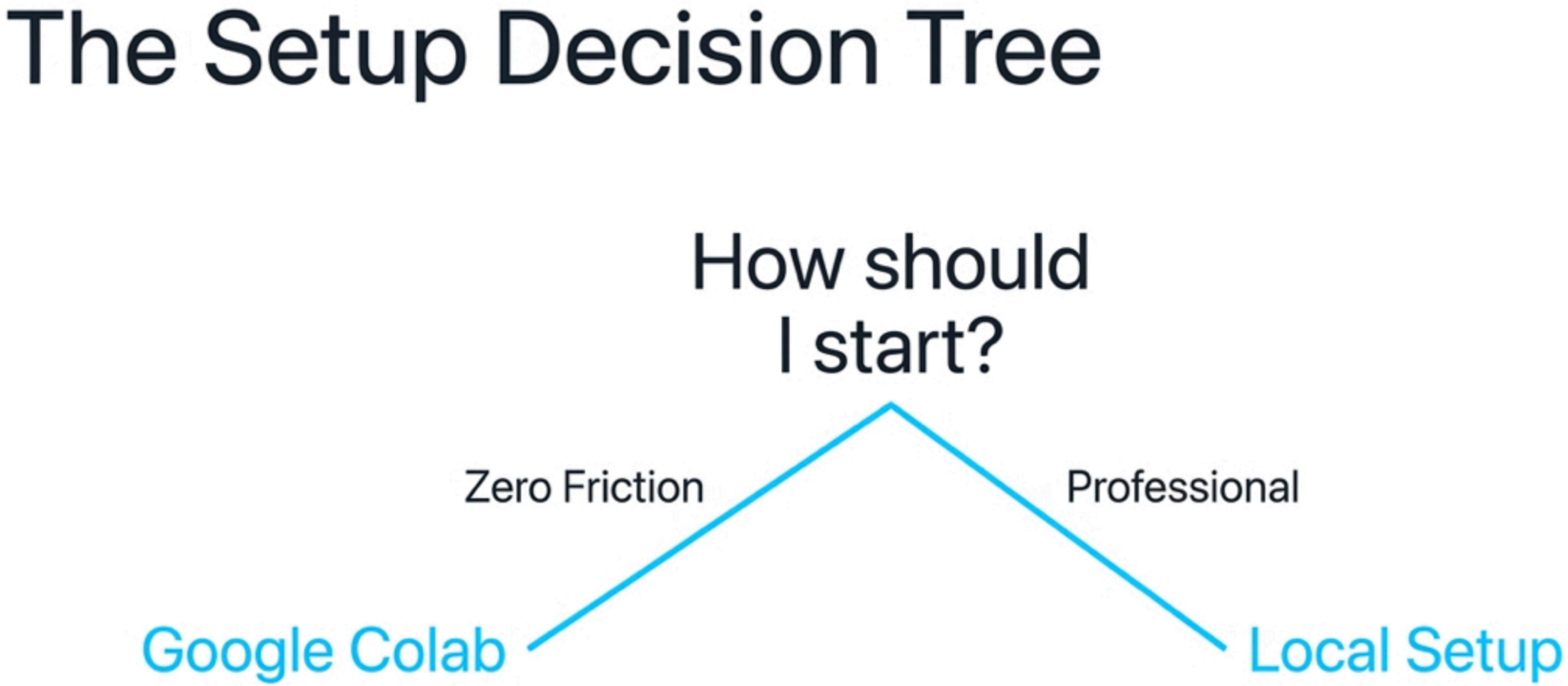
1.3.2 現代 Python 開發工具總覽（含 AI 時代工具）

工欲善其事，必先利其器。現代 Python 開發工具大致可分為三大類：



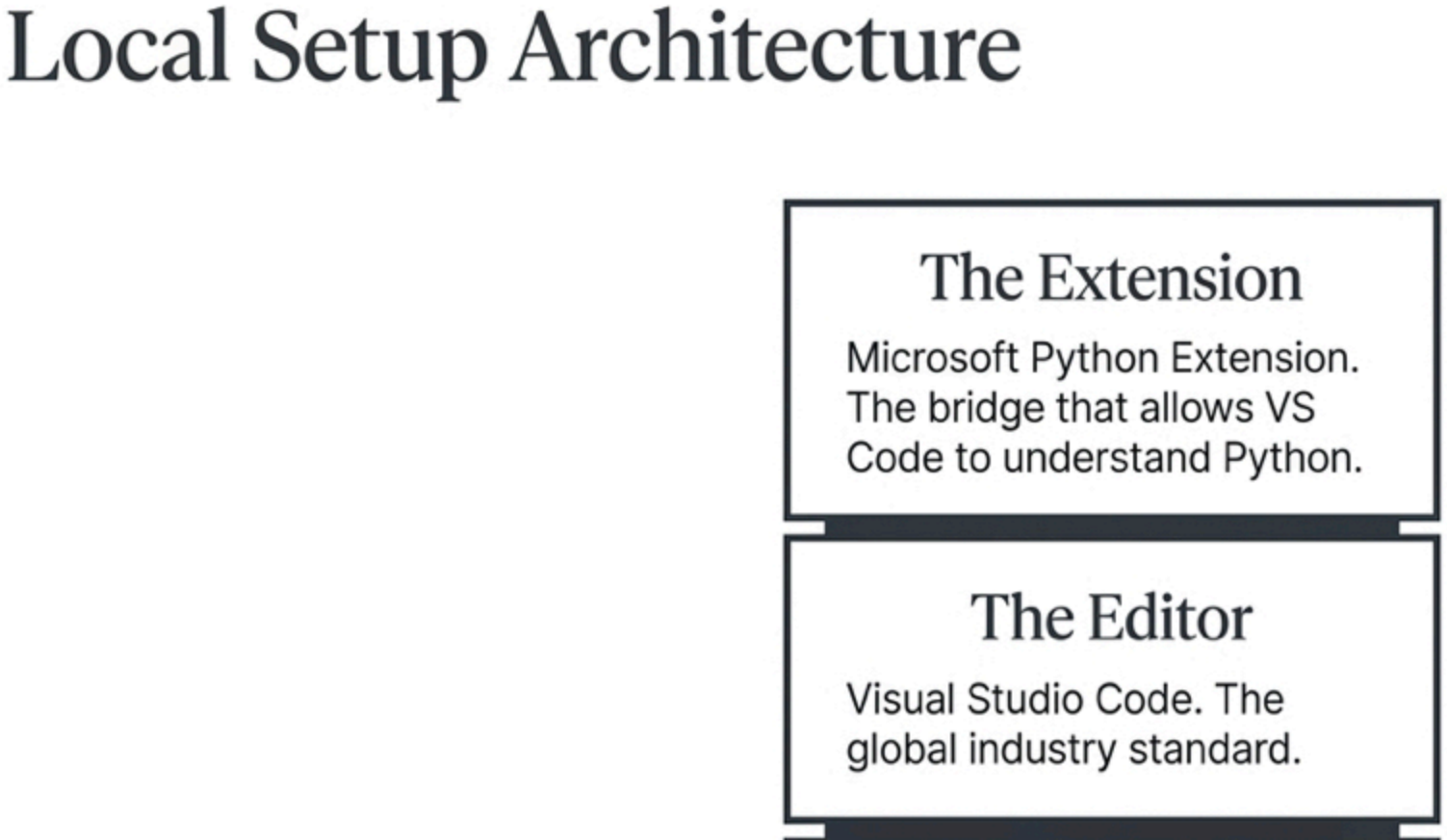
1.3.3 初學者的兩條安裝路徑

對於初學者來說，有兩種主要的路徑可以選擇：**使用線上開發環境**或**在自己的電腦上安裝**。



路徑二：在本機電腦安裝 (專業開發的必經之路)

當你熟悉了基本語法後，建立本地開發環境是讓你成為更專業開發者的下一步。我們的推薦組合是 **Python 官方核心 + Visual Studio Code (VS Code) 編輯器 + Microsoft Python 擴充套件。**



第一階段：安裝或更新 Python 核心程式

若檢查後發現需要安裝或更新，請依照以下步驟操作：

1. **Windows 系統安裝指南**：
    - **下載 Python**：前往 Python 官方網站 https://www.python.org，將滑鼠移到 `Downloads` 選單上，點擊 `Download for Windows` 下的 **Python [最新版本號]** 按鈕。
    - **執行安裝程式**：到「下載」資料夾，雙擊執行剛下載的 `python-[...].exe` 安裝檔。
    - **關鍵設定（極重要）**：在安裝程式的第一個畫面，**務必勾選左下角的「Add python.exe to PATH」選項！** 這個步驟能確保你在系統任何路徑下都能順利執行 Python 指令。勾選後點擊 `Install Now`。
    - **完成安裝**：等待進度條跑完，看到 "Setup was successful" 畫面即表示安裝成功。
2. **macOS 系統安裝指南**：
    - **下載 Python**：前往 Python 官方網站 https://www.python.org，將滑鼠移到 `Downloads` 選單上，點擊 `Download for macOS` 下的 **Python [最新版本號]** 按鈕。
    - **執行安裝程式**：到「下載項目」資料夾，雙擊執行 `python-[...].pkg` 安裝套件。

第二階段：安裝與設定 Visual Studio Code (VS Code)

現在你的電腦已經擁有 Python 核心了，接下來我們要安裝一套現代強大的程式碼編輯器：

1. **下載並安裝 VS Code**：
    - 前往 VS Code 官方網站 https://code.visualstudio.com。
    - 網站會自動偵測你的作業系統，直接點擊藍色的下載按鈕。下載完成後按照預設選項完成安裝。
2. **安裝 Python 擴充套件（關鍵步驟）**：
    - 打開安裝好的 VS Code。
    - 點擊左側活動列中的「**擴充功能 (Extensions)**」圖示（四個方塊圖示，或按快捷鍵 `Ctrl+Shift+X` / `Cmd+Shift+X`）。
    - 在頂端搜尋框輸入 `Python`。
    - 找到由 **Microsoft** 發行的官方套件，點擊 `Install` 安裝。
3. **撰寫並執行你的第一支本地程式**：

1.3.4 Python 虛擬環境 (Virtual Environments) 基礎概念

在日後的開發專案中，不同專案往往會依賴不同版本的第三方套件（例如專案 A 需要 Pandas 1.5，而專案 B 需要 Pandas 2.2）。如果全部安裝在全域環境中，將會引發嚴重的版本衝突災難。

Python 內建的 **`venv` 模組** 允許我們為每個專案建立獨立、互不干擾的沙盒環境：

```
# 1. 在專案資料夾下建立名為 .venv 的虛擬環境
python3 -m venv .venv

# 2. 啟用虛擬環境
# macOS / Linux 系統：
source .venv/bin/activate

# Windows 系統 (PowerShell)：
.venv\Scripts\Activate.ps1

# 3. 啟用後，終端機提示字元前會出現 (.venv)，此時安裝的套件只會存在此專案內
pip install pandas requests
```

1.3.5 隨堂測驗 (CCQ 2)

問題

在 Windows 系統安裝 Python 官方安裝檔時，如果遺漏勾選了「Add python.exe to PATH」選項，後續在命令提示字元 (cmd) 中輸入 `python` 指令時，最常遇到什麼問題？

A) 電腦螢幕解析度會被自動調降。
B) 系統會顯示「'python' 不是內部或外部命令、可執行的程式或批次檔」，因為作業系統不知道去哪個資料夾路徑尋找 `python.exe`。
C) 安裝程式會自動格式化硬碟。
D) Python 程式碼中的字串會全部變成亂碼。

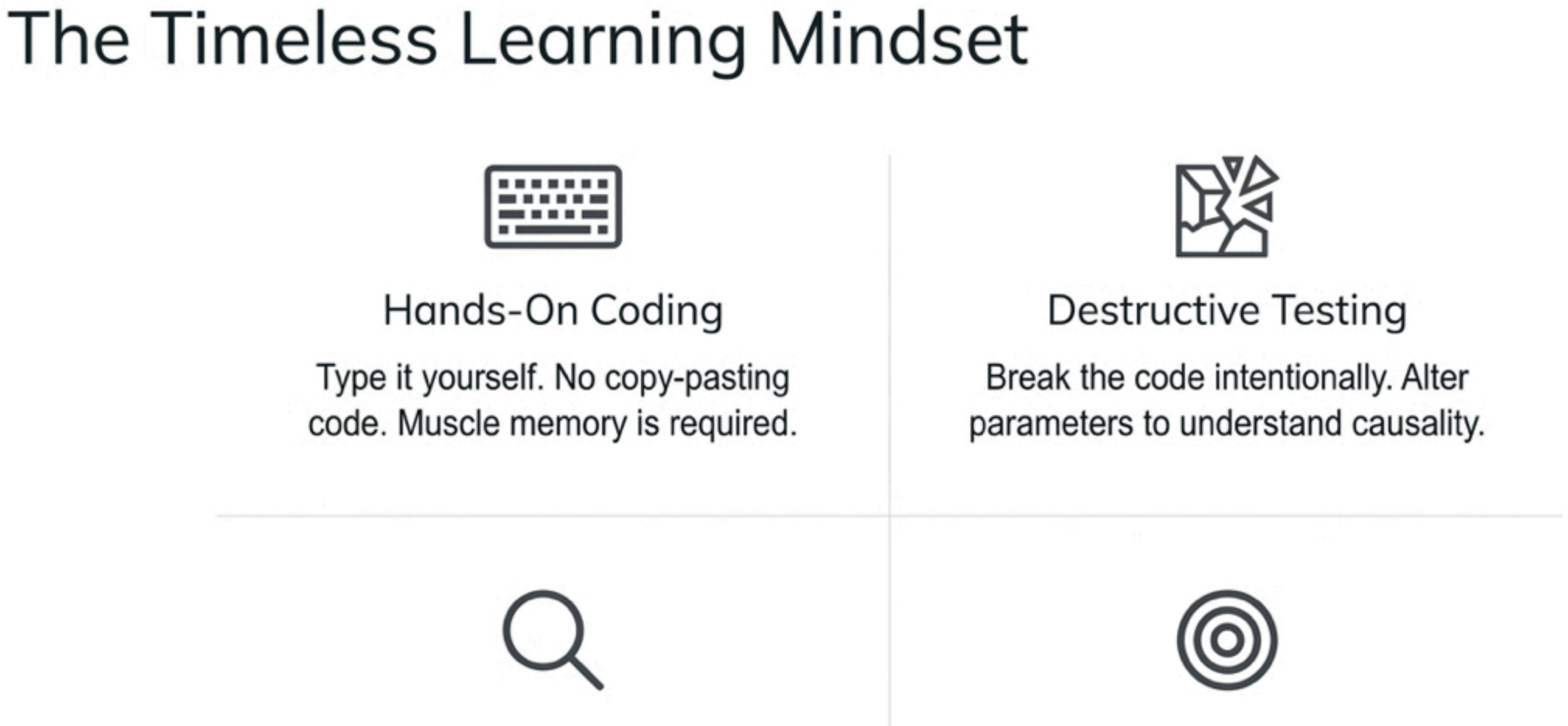
▶ 點擊查看【隨堂測驗】答案與解析

1.4 給學習者的建議

學寫程式就像學任何一門技藝，正確的方法與心態至關重要。本節將分為兩部分：首先是所有程式學習者都應具備的通用心法；接著是針對當前 AI 世代，如何善用新工具的智慧學習策略。

1.4.1 通用的程式學習心法

這些是歷久不衰的學習原則，無論科技如何演進，它們都是建立扎實程式能力的基石。



1.4.2 AI 世代的智慧學習策略

在 AI 工具普及的今天，學習方式正在發生革命性的變化。聰明的學習者不再是從零開始，而是站在 AI 的肩膀上，將精力專注於更高層次的思考。



1.4.3 初學者的 5 大常見「撞牆」陷阱與避坑指南

根據教學經驗，初學者在撰寫 Python 程式的前兩週，有超過 80% 的挫折感來自於以下 5 個極其微小但致命的語法陷阱。提早認識它們，能為你省下數十個小時無謂的除錯時間：

1. **大小寫極度敏感 (Case Sensitivity)**：
    - Python 對於英文字母的大小寫區分非常嚴格。
    - ❌ 錯誤寫法：`Print("Hi")`、`TRUE`、`false`
    - ✅ 正確寫法：`print("Hi")`、`True`、`False`

2. **中英文全形/半形標點符號混淆 (Full-width Punctuation)**：
    - 這是中文使用者最常犯的隱形錯誤。Python 只認得**半形英文標點符號**。
    - ❌ 致命符號：中文冒號 `：`、中文引號 `”`、中文逗號 `，`、中文括號 `（）`
    - 💡 **解法**：在寫程式時，請養成將輸入法切換為「純英文半形 (ENG)」的習慣。

3. **縮排不一致 (IndentationError)**：

1.5 善用線上資源，打造你的個人化學習路徑

數位時代賦予了我們前所未有的學習自由。線上自學已不僅是一種趨勢，更是現代人提升自我、保持競爭力的核心技能。它讓我們能打破傳統教育的框架，根據自身的需求與步調來建構知識。

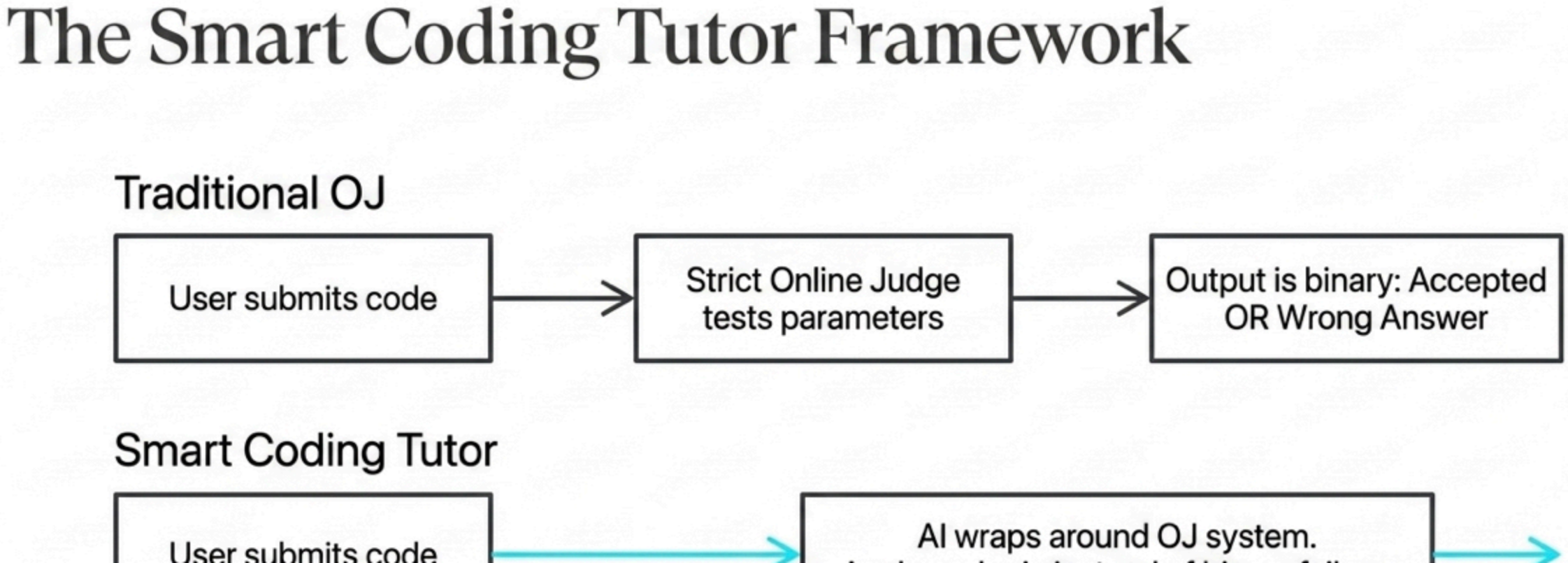
- **極致的學習彈性**：你不僅可以自由安排學習的時間與地點，更能實現「精通式學習」(Mastery-based learning)。遇到不懂的概念，可以反覆觀看影片、查閱資料，直到完全理解為止，而不用被固定的課程進度推著走。你也可以利用通勤、午休等零碎時間進行「微學習」(Micro-learning)，完成一個小單元或解一道練習題。

- **無盡的資源寶庫**：整個網際網路都是你的圖書館。從頂尖大學在 Coursera、edX 等平台開設的系統化公開課 (MOOCs)，到 YouTube 上無數的程式設計教學影片與直播，再到 LeetCode、Codewars 等互動式程式練習平台，你可以接觸到全世界最新、最優質的教育資源。

- **終身學習的實踐場**：科技領域的知識迭代速度極快，一個今天熱門的框架，可能在幾年後就被取代。持續學習不是一個選項，而是專業人士的日常。線上資源讓「技能更新」(Upskilling) 與「技能轉型」(Reskilling) 變得觸手可及，成為職涯發展中最強大的推進器。

1.5.1 AI 學習助教：Smart Coding Tutor

我們理解，純粹的線上自學有時會讓人感到孤獨與挫折，尤其是在程式卡關、求助無門的時候。為了解決這個痛點，本課程提供了一個結合「線上解題系統 (Online Judge)」與「AI 輔助」的獨特學習環境 —— **Smart Coding Tutor**。



1.5.2 經典 Python 實作演練：Hello World 與互動式終端 (REPL)

現在，讓我們親自體驗 Python 最令人著迷的特性之一——**REPL (Read-Eval-Print Loop) 互動式直譯環境**：

1. 打開終端機 (macOS Terminal 或 Windows cmd)，輸入 `python` (或 `python3`) 並按下 Enter：

   ```
   $ python3
   Python 3.12.2 (main, Feb 20 2026, 10:00:00)
   [Clang 15.0.0 (clang-1500.1.0.2.5)] on darwin
   Type "help", "copyright", "credits" or "license" for more information.
   >>>
   ```

2. 看到三個大於符號 `>>>` 即代表進入了 REPL 環境！你可以直接輸入 Python 表達式，直譯器會「即讀、即算、即印」：

   ```
   >>> print("Hello, World!")
   Hello, World!

   >>> 100 * 3 + 25
   ```

1.5.3 隨堂測驗 (CCQ 3)

問題

身為 Python 初學者，若你撰寫了一行程式碼 `Print("歡迎學習 Python")`，在執行時系統回報了 `NameError: name 'Print' is not defined`。這項錯誤發生的最主要原因為何？

A) 電腦尚未連接網際網路，無法下載字型檔。
B) Python 的函式名稱對英文大小寫極度敏感，內建的輸出函式是全小寫的 `print`，大寫開頭的 `Print` 會被視為未宣告的變數。
C) 字串必須用三個雙引號包覆才合法。
D) Python 不支援在字串中印出繁體中文字元。

▶ 點擊查看【隨堂測驗】答案與解析